

visión general.



El sistema tiene **5 capas funcionales** (captura → ASR → NLP clínico → LLM → revisión/exportación) más una franja transversal de requisitos legales y no funcionales que aplican a todo el sistema.

Interfaces entre capas — cada flecha es un contrato de datos. La salida del ASR (texto + timestamps + confianza) es la entrada del NLP, que a su vez produce entidades estructuradas para el LLM. Habrá que definir estos esquemas intermedios.

Modularidad del ASR — Whisper u otros. Patrón adaptador/estrategia en esa capa.

Capa de validación FHIR — entre el LLM y la exportación conviene añadir un validador (p.ej. con el IG de HL7 España) antes de que el médico revise, no después.

Obtención y envío del audio

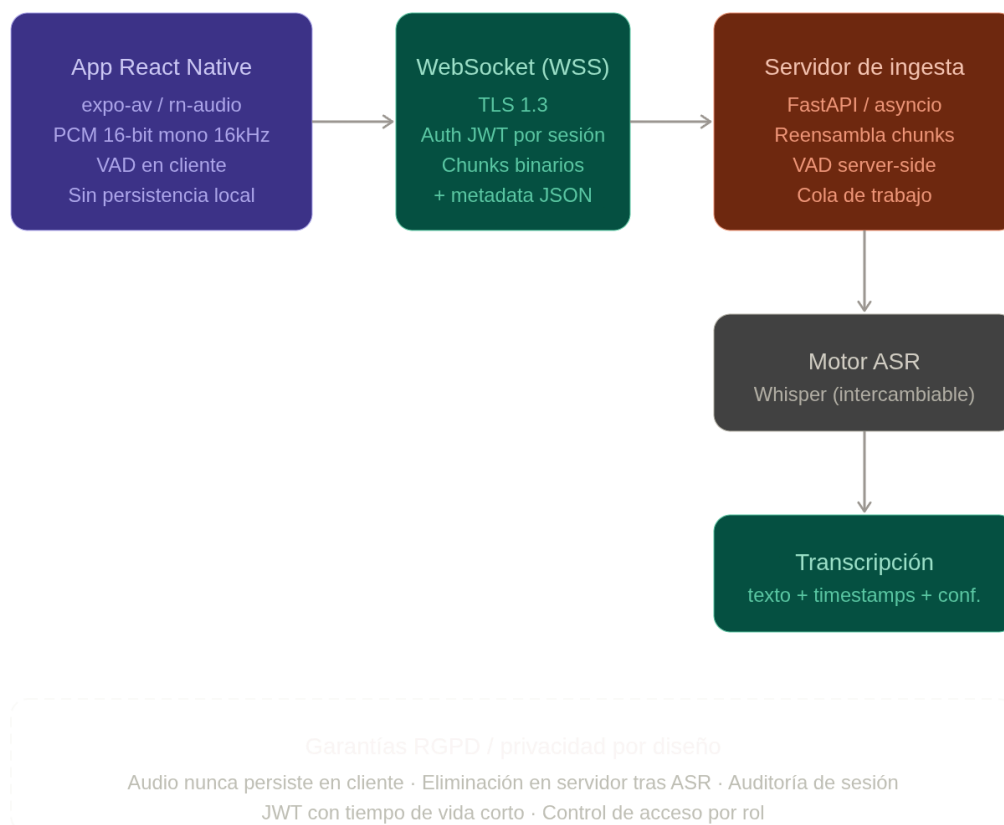
React Native + grabación de audio Sin problema. La librería `expo-av` o `react-native-audio-recorder-player` permiten grabar y emitir chunks. El formato recomendado para Whisper es PCM 16-bit mono 16kHz — conviene fijarlo desde el cliente para no transcodificar en el servidor.

Socket vs MQTT MQTT está diseñado para dispositivos IoT con conexiones inestables y mensajes pequeños y frecuentes. Para audio en una consulta (red WiFi estable, sesiones de 1-3 min, payloads de varios KB) **WebSocket es más natural**: menos overhead de broker, librerías más maduras en React Native, y el backend lo maneja directamente sin infraestructura adicional.

Chunking cada X segundos (tu propuesta) vs por silencio (VAD) El problema de chunks por tiempo fijo es que puedes cortar a la mitad de una palabra, lo que penaliza el WER. Con entorno silencioso tienes la opción de hacer VAD (*Voice Activity Detection*) ligero en el cliente — Whisper lo soporta internamente, o puedes usar `webrtcvad` en el servidor — y enviar chunks al detectar una pausa natural (final de frase). Esto mejora la transcripción sin añadir latencia perceptible para el médico (que ya dijiste que no le importa).

Privacidad / RGPD Como el audio contiene datos de salud, hay dos puntos críticos:

- El chunk nunca debe persistir en disco en el cliente.
- La transmisión debe ir sobre TLS (WSS, no WS).
- En el servidor, el audio debe eliminarse en cuanto se transcribe, guardando solo el texto.



MQTT → WebSocket (WSS). Mismo resultado, menos infraestructura. Sin broker, sin topics, sin QoS levels que gestionar. Con un entorno de consulta y WiFi estable no necesitas la robustez de MQTT.

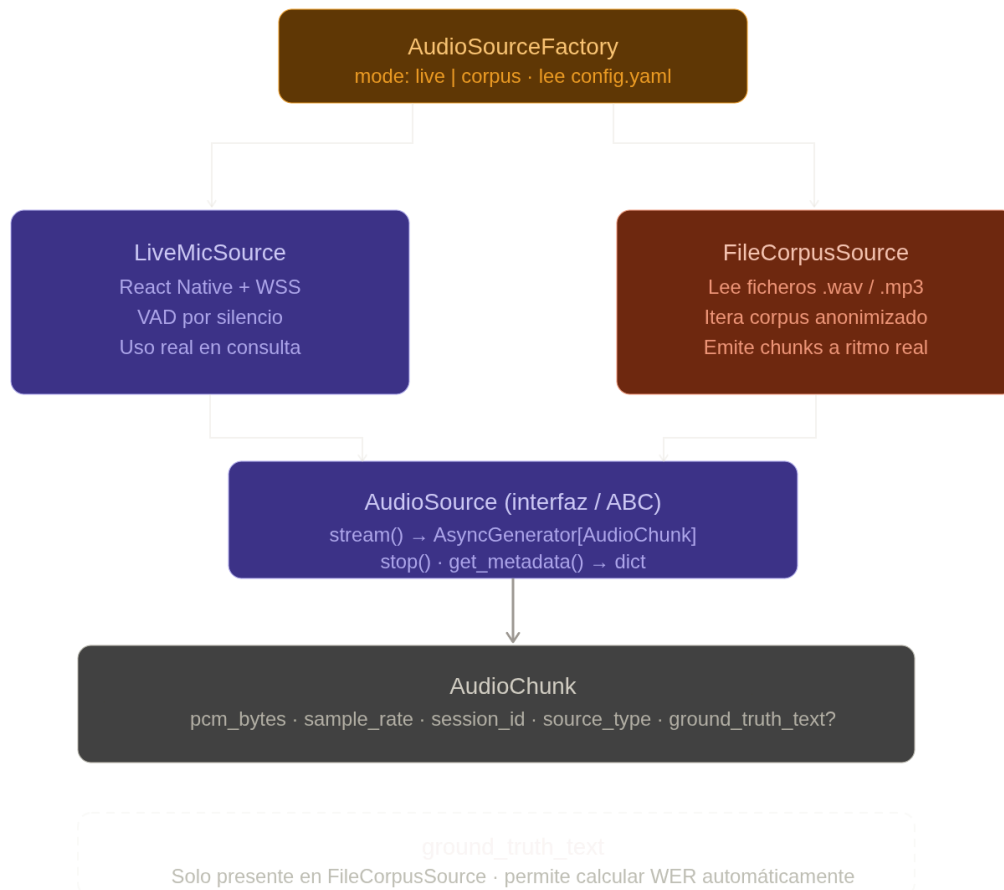
Chunking por tiempo → chunking por silencio (VAD). Con entorno sin ruido el VAD es trivial: un umbral de energía en el cliente ya es suficiente para detectar pausas naturales entre frases. Evitas cortar palabras a la mitad y el WER mejora sin coste.

Un detalle que no aparece en tu propuesta original pero que es importante para el TFM: el chunk debe llevar metadata JSON junto al binario de audio (id de sesión, número de secuencia, timestamp), para que el servidor pueda reensamblar correctamente si algún chunk llega desordenado.

Reconocimiento del lenguaje natural

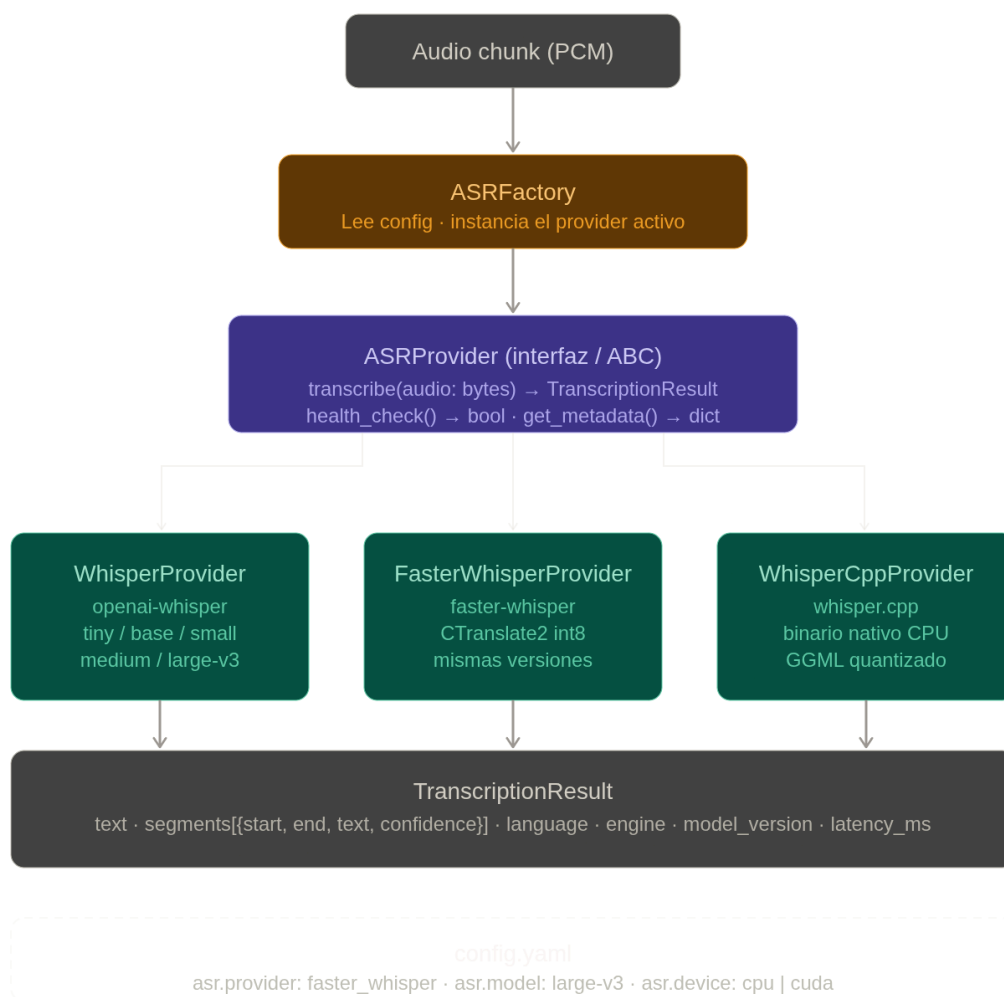
Se seleccionan tres motores de la misma familia Whisper es ideal para el TFM: puedes aislar la variable "implementación/optimización" manteniendo el modelo base constante, lo que da comparativas muy limpias (WER, latencia, uso de RAM/VRAM, requisitos de hardware). La orquestación por configuración es la más pragmática para un TFM: un fichero `config.yaml` o variable de entorno selecciona el motor activo, sin complejidad de paralelismo.

Se quiere poder utilizar tanto la voz de la persona como poder utilizar audios anonimizados como input del sistema. Por lo que se utiliza un



El proceso empieza leyendo la configuración del `config.yaml`,

FileCorpusSource tiene un detalle importante: **emite chunks a ritmo real** (simulando la velocidad de dictado natural) en lugar de enviar el fichero entero de golpe configurable desde el fichero de config. Así las métricas de latencia del ASR son comparables entre el modo corpus y el modo live.



El contrato **ASRProvider** define solo tres métodos. `transcribe()` es el núcleo; `health_check()` permite verificar que el modelo está cargado antes de recibir audio; y `get_metadata()` devuelve nombre del motor, versión del modelo y dispositivo — útil para el sistema de métricas del TFM.

TranscriptionResult es el contrato de salida. Todos los providers devuelven exactamente lo mismo, incluyendo `engine` y `model_version`. Así la capa NLP no sabe ni le importa qué motor generó la transcripción, y el módulo de evaluación puede agrupar resultados por motor automáticamente.

ASRFactory lee el `config.yaml` e instancia el provider correcto. Un cambio de una línea en config es suficiente para cambiar de motor en todos los experimentos. Tiene sentido también que la factory precargue el modelo en memoria al arrancar el servidor, no en cada petición.

Para las métricas del TFM, el campo `latency_ms` en **TranscriptionResult** junto con los logs del servidor te da WER, latencia de transcripción y uso de recursos por motor de

forma casi automática. Podrías añadir un módulo **ASREvaluator** que consuma esos resultados y genere los informes comparativos.

¿Seguimos con la capa de extracción clínica (NLP), o quieres revisar primero cómo modelar el módulo de evaluación/métricas que atraviesa todo el pipeline?

You said: me gustaria agregar algo al primer paso.

me gustaria agregar algo al primer paso. Actualmente cuento con algunos audios medicos anonimizados con los cuales me gustaria poner a prueba que tan buenos son los modelos de reconocimiento de lenguaje natural. Por lo que debemos pensar una solucion (capaz colocando alguien en medio que simule ser una persona) para poder trabajar tanto con mi voz como la de audios medicos

8:05 PM

Claude responded: Buena idea, y es un requisito real del TFM: necesitas poder evaluar los modelos con corpus controlado (tus audios anonimizados) además de con dictado en vivo.

Buena idea, y es un requisito real del TFM: necesitas poder evaluar los modelos con corpus controlado (tus audios anonimizados) además de con dictado en vivo. Esto se resuelve limpiamente con un segundo "provider" en la capa de entrada, no tocando nada del pipeline ASR.

La idea es que **ASRFactory** no sepa si el audio viene del micrófono o de un fichero. Quien cambia es la **fuentes de audio**, no el motor.

v

visualize

v

visualize show_widget

El cambio clave está en **AudioChunk**: tiene un campo opcional **ground_truth_text** que solo se rellena cuando la fuente es **FileCorpusSource**. El pipeline ASR no cambia en absoluto — simplemente ignora ese campo. Pero el módulo de evaluación lo usa para calcular el WER automáticamente comparando la transcripción del motor con el texto de referencia.

FileCorpusSource tiene un detalle importante: **emite chunks a ritmo real** (simulando la velocidad de dictado natural) en lugar de enviar el fichero entero de golpe. Así las métricas de latencia del ASR son comparables entre el modo corpus y el modo live.

La estructura de ficheros del corpus quedaría algo así:

None

corpus/

audio_001.wav

audio_001.txt ← ground truth

audio_002.wav

audio_002.txt

manifest.json ← metadatos: especialidad, duración, locutor_id
anonimizado

El `manifest.json` te permite luego segmentar los resultados por especialidad o duración del dictado en los informes comparativos del TFM.

¿Seguimos con la capa de extracción clínica, o quieres ver primero cómo encajaría el módulo de evaluación/métricas que conecta ambas capas?